El Durazno
Del HTML al primer backend

GUÍA DE ESTUDIO · ACTUALIZADA AL 27 DE OCTUBRE DE 2026

Un repaso del proyecto real, organizado para leer en el teléfono y acompañar el audio. Incluye qué hicimos, por qué, ejemplos de código y preguntas para comprobar lo aprendido.

Qué está funcionando

Página adaptable, mapa, servicios y preguntas frecuentes. Calendario con selección de fechas, reglas de noches y reservas de prueba obtenidas desde el servidor Express.

Qué sigue pendiente

Base de datos, creación y confirmación de reservas reales, validación decisiva en el servidor, control de concurrencia, panel privado y pagos. El formulario de contacto aún no envía mensajes.

Cómo usar esta guía

Leé un tema por vez. Antes de mirar el código, explicá qué problema resuelve. Contestá la pregunta al final de cada tema y después probá un caso en tu proyecto.

Contenido

01 / El proyecto y su estado

02 / HTML: la estructura

03 / Navegar, mostrar y describir

04 / CSS: una composición coherente

05 / JavaScript y los elementos

06 / Fechas y noches

07 / Evitar superposiciones

08 / El calendario visual

09 / Git, npm y dependencias

10 / El primer servidor

11 / De la API al calendario

12 / Repasar y seguir

01 / El proyecto y su estado

El Durazno es una web para un alojamiento familiar en Andacollo. La meta es mostrar la cabaña, consultar fechas y, más adelante, gestionar reservas desde un servidor con base de datos. El desarrollo se está usando también para aprender, con especial interés en backend.

Hoy funcionan la página pública, la presentación adaptable, el mapa, las preguntas frecuentes y un calendario que recibe dos reservas ficticias desde Express. Elegir fechas calcula noches y comprueba superposiciones. No crea reservas reales.

La separación clave es: HTML describe la estructura; CSS organiza la apariencia; JavaScript agrega comportamiento. El navegador ejecuta el frontend. Node.js ejecuta el servidor. Una API permite que ambos intercambien datos.

La base de datos, la confirmación de reservas, el panel privado, la autenticación y los pagos todavía están pendientes. El formulario de contacto tampoco envía mensajes. Las condiciones de seña y cancelación aún requieren algunas definiciones familiares.

Para recordar

Recibir datos del servidor no equivale a tener una base de datos ni un sistema de reservas completo.

Comprobá que lo entendiste

¿Qué puede hacer hoy un visitante y qué partes siguen siendo una demostración?

02 / HTML: la estructura

Empezamos con un documento HTML en español. En head están los metadatos, el título de la pestaña, el enlace al CSS y los archivos JavaScript. En body está el contenido que ve la persona. head y header son etiquetas diferentes.

header agrupa la cabecera y el menú. nav identifica la navegación. main contiene el contenido principal. Cada section reúne un tema, como Alojamiento o Ubicación. footer contiene el pie de página.

El h1 es el título principal; los h2 identifican secciones; los h3, preguntas dentro de Preguntas frecuentes. Un h3 tiene menor jerarquía aunque lleve la misma sangría que un h2. La sangría muestra anidamiento; el número del encabezado expresa su jerarquía.

Si abrís una etiqueta dentro de otra, cerrás primero la interior. Un div sirve para agrupar contenido cuando hace falta, como el título y la descripción de Alojamiento. Una clase permite identificar ese grupo para darle estilos.

```
<section id="alojamiento">
    <h2>Alojamiento</h2>
    <p>Descripción de la cabaña.</p>
</section>
```

Para recordar

La etiqueta que abriste última se cierra primero. Las etiquetas hermanas se alinean.

Comprobá que lo entendiste

¿Por qué el título h2 y una pregunta h3 pueden tener los mismos espacios delante?

03 / Navegar, mostrar y describir

Un enlace con href numeral alojamiento lleva al elemento cuyo id es alojamiento. El numeral está en el enlace; no forma parte del identificador. Cada id debe ser único en la página.

La imagen usa src para indicar el archivo y alt para describir su contenido. El nombre debe coincidir con exactitud: frente espacio uno, punto jpeg, no frente uno sin espacio ni punto jpg. La extensión y la ruta forman parte del nombre.

alt permite comunicar la información a una persona que usa lector de pantalla y ofrece una alternativa cuando la imagen no carga. Por eso los pictogramas tienen una descripción de baño, ducha, wifi, cinco personas, estacionamiento y parrilla.

El mapa se incorpora mediante iframe. La etiqueta de apertura puede ocupar varias líneas: termina en el signo mayor que después del último atributo. El cierre es otra etiqueta. Su title identifica el mapa para tecnologías de asistencia.

Los textos y servicios se redactaron con información de la familia. No se publican características no confirmadas: por ejemplo, actualmente no hay televisión. El mapa y las fotos ayudan a presentar el alojamiento, pero no calculan disponibilidad.

```
<a href="#alojamiento">Ver alojamiento</a>
<img src="imagenes/frente 1.jpeg"
     alt="Frente de la cabaña El Durazno">
```

Para recordar

HTML incorpora la imagen; CSS cambia su apariencia. Una regla de CSS no crea por sí sola la foto.

Comprobá que lo entendiste

¿Qué revisarías si una foto no aparece, aunque el CSS parezca correcto?

04 / CSS: una composición coherente

La referencia visual aportó naturaleza, fotos protagonistas y espacio entre bloques. La primera versión se sentía demasiado básica; después adoptamos fondo claro, verde oscuro y títulos tradicionales, con texto sobre fondos lisos.

Una regla CSS tiene un selector y propiedades. El selector puede ser una etiqueta, un id precedido por numeral, o una clase precedida por punto. La clase servicios permite cambiar solo el tamaño de los pictogramas.

margin separa elementos por fuera; padding agrega espacio interior. max-width limita el ancho. height auto mantiene la proporción de una imagen. box-sizing border-box incluye el borde y el relleno dentro del ancho declarado.

Grid permite distribuir contenido en columnas. Para evitar el hueco entre el título de Alojamiento y su descripción, los agrupamos y dejamos los pictogramas en la otra columna. En pantallas pequeñas los bloques se apilan mediante media queries.

La revisión general unificó anchos, encabezados y espacios entre secciones. El calendario usa los mismos colores y tipografías. Quedó pendiente reforzar visualmente la diferencia entre fechas libres, ocupadas y seleccionadas; no hicimos ese cambio en esta etapa.

```
.servicios {
    width: 100%;
    max-width: 360px;
    height: auto;
    margin: 0 auto;
}
```

Para recordar

Consistencia significa repetir criterios de alineación, espacio y tipografía, no hacer idénticas todas las secciones.

Comprobá que lo entendiste

¿Qué diferencia hay entre el espacio del código y los márgenes que ve el visitante?

05 / JavaScript y los elementos

Conectamos JavaScript en head mediante script con defer. El navegador descarga el archivo mientras procesa el documento y lo ejecuta después de analizar el HTML. El script de la página se conecta una sola vez.

document representa la página y getElementById busca un elemento. Una constante puede guardar la referencia a ese elemento. const impide reasignar la variable, pero no congela el campo ni su contenido. let permite una asignación posterior, como reemplazar una lista vacía por los datos del servidor.

Al principio usamos dos campos de fecha y un formulario. addEventListener escuchaba submit. preventDefault evitaba el envío normal, que recargaba la página. textContent escribía la respuesta. value permitía leer las fechas elegidas.

Esa fue una etapa de aprendizaje. Ahora los campos manuales y el botón de consulta fueron retirados: se elige directamente en el calendario y el resumen aparece automáticamente. La selección se guarda en un objeto con entrada y salida.

Los nombres del código pueden estar en inglés para seguir convenciones habituales. Los mensajes para huéspedes siguen en español. Los nombres son libres; los métodos del lenguaje deben escribirse exactamente.

```
const selection = {
    checkIn: "",
    checkOut: ""
};

let reservations = [];
```

Para recordar

No confundas una referencia al elemento con su valor. Tampoco confundas const con un objeto inmutable.

Comprobá que lo entendiste

¿Por qué usamos let para la lista recibida del servidor?

06 / Fechas y noches

Una estadía necesita fechas completas y una salida posterior a la entrada. Si falta una fecha o el orden es incorrecto, se muestra un mensaje y se detiene esa comprobación. if ejecuta código cuando una condición se cumple; return termina la función.

Los campos iniciales entregaban textos con formato año, mes y día, separados por guiones. Para fechas válidas y completas, ese formato permite comparar el orden como texto. La lógica actual también valida que la fecha exista, para rechazar casos como el treinta de febrero.

El rango se mueve con el día: desde hoy hasta el mismo día del año siguiente. No equivale a sumar siempre trescientos sesenta y cinco días. Si hoy fuera veintinueve de febrero y el año siguiente no fuera bisiesto, el máximo sería veintiocho de febrero.

Para contar noches restamos dos fechas interpretadas en UTC y dividimos por los milisegundos de un día. Esto evita errores por cambios de horario local en esa resta. Los límites de hoy, en cambio, parten de la fecha local del navegador.

Del quince al dieciocho hay tres noches: quince, dieciséis y diecisiete. El dieciocho es la salida. No se suma una noche por el hecho de mencionar cuatro fechas del calendario.

```
const millisecondsPerDay = 24 * 60 * 60 * 1000;
const nights =
    (Date.parse(checkOut) - Date.parse(checkIn))
    / millisecondsPerDay;
```

Para recordar

La entrada incluye su noche; la salida no la ocupa. Los límites se actualizan al abrir y al volver a usar el calendario.

Comprobá que lo entendiste

¿Cuántas noches hay del treinta y uno de diciembre al primero de enero?

07 / Evitar superposiciones

Una reserva del diez al quince ocupa las noches del diez al catorce. Otra del doce al dieciséis comparte noches y debe rechazarse. Una del quince al dieciocho puede empezar cuando termina la anterior.

La condición de conflicto combina dos preguntas: ¿la entrada nueva ocurre antes de la salida existente? ¿La salida nueva ocurre después de la entrada existente? El operador doble ampersand exige que ambas sean verdaderas.

Comparar solo si una estadía queda enteramente dentro de otra era insuficiente: dejaba pasar conflictos parciales o reservas que cubren todo el período. Las desigualdades estrictas permiten el cambio de huésped en una misma fecha.

Primero simulamos una reserva con fechas fijas. Después creamos una lista con varias. Los corchetes contienen la lista; las llaves agrupan los datos de cada reserva. Cada objeto tiene su entrada y su salida.

for of recorre la lista. En cada vuelta, la variable reservation representa un objeto diferente. El if se escribe dentro del recorrido. Los cierres deben quedar dentro de la función que usa las fechas. Ahora esas reglas se reúnen en availability.js y sirven al calendario.

```
checkIn < reservation.checkOut &&
checkOut > reservation.checkIn
```

Para recordar

Validar datos primero; comprobar conflictos después. Una entrada posterior también puede ser válida: no hace falta llegar exactamente el día de salida anterior.

Comprobá que lo entendiste

¿Por qué del ocho al diecisiete también choca con una reserva del diez al quince?

08 / El calendario visual

El calendario convirtió la selección de fechas en una interacción visible. Muestra el mes y año, días de la semana, noches ocupadas y fechas seleccionadas. Las flechas permiten recorrer los meses dentro del límite permitido.

El primer clic elige entrada. El segundo elige salida si el rango es válido. Una nueva entrada después de completar la selección permite comenzar otra consulta. Borrar fechas limpia la selección y el resumen.

Los días pasados y las entradas imposibles no se pueden elegir. Si una noche está ocupada, normalmente no permite empezar allí. Pero esa misma fecha puede servir de salida si es el comienzo de otra reserva y no se comparte ninguna noche.

Esa distinción es importante: una noche ocupada no siempre prohíbe salir ese día. El calendario y la validación usan la misma regla para no contradecirse. También impiden elegir una salida que salte por encima de noches ocupadas.

Los botones tienen descripciones de fecha y estado para lectores de pantalla, y pueden utilizarse con teclado. El resumen comunica entrada, salida y noches sin exigir volver a escribir fechas. Los datos aún son de demostración.

Para recordar

La interfaz ayuda a elegir; las reglas determinan si la estadía es válida. Una futura reserva real requerirá validación del servidor.

Comprobá que lo entendiste

¿Puede una persona salir el veinte si ese día comienza una reserva del veinte al veinticinco?

09 / Git, npm y dependencias

Git guarda versiones del proyecto mediante commits. Revisar el estado permite saber qué cambió antes de guardarlo. Un aviso sobre LF y CRLF describe dos convenciones de saltos de línea; no significa que el CSS esté roto.

Node.js ejecuta JavaScript fuera del navegador. npm administra paquetes del proyecto. npm init con la opción de aceptar valores iniciales creó package.json. Después instalamos Express, que quedó registrado como dependencia.

package-lock.json guarda las versiones resueltas de las dependencias. node_modules contiene los paquetes descargados. Esa carpeta se puede reconstruir mediante npm; por eso agregamos node_modules barra al archivo .gitignore.

El archivo .gitignore va en la carpeta principal, junto a package.json. No va dentro de node_modules. Es un archivo de texto con ese nombre exacto, sin punto txt. El propio .gitignore y los dos archivos de configuración sí se guardan en Git.

Ignorar un archivo no lo borra ni impide usarlo. Además, si un archivo ya estaba bajo seguimiento, agregarlo después a .gitignore no lo saca automáticamente del historial. Por ahora usamos la exclusión para las dependencias recién instaladas.

```
npm init -y
npm install express

# En .gitignore:
node_modules/
```

Para recordar

Servidor y navegador ejecutan JavaScript, pero cumplen tareas diferentes. npm instala herramientas; Git guarda el historial.

Comprobá que lo entendiste

¿Qué archivos permiten que otra persona reconstruya las dependencias sin copiar node_modules?

10 / El primer servidor

Creamos server.js y cargamos Express con require. express crea la aplicación. app.get registra una dirección y una función que responde cuando llega una petición GET.

req representa la petición y res la respuesta. Al principio la dirección principal enviaba un texto con res.send. Ver ese mensaje en el navegador confirmó que la petición llegó al servidor y volvió una respuesta.

Después agregamos la dirección de API para reservas. res.json devuelve la lista como JSON, un formato de intercambio de datos. Las dos reservas de noviembre viven en memoria dentro del código del servidor. Todavía no están en una base de datos.

La dirección principal ahora entrega el HTML mediante sendFile. Otras rutas sirven CSS y JavaScript, y una carpeta específica entrega las imágenes. dirname identifica la carpeta del servidor. No se publica indiscriminadamente toda la carpeta del proyecto.

app.listen inicia el proceso en el puerto tres mil de esta computadora. Para detenerlo se usa Control C en la terminal. Al cambiar server.js hay que reiniciar Node con la configuración actual. El archivo del servidor no se agrega al HTML.

```
app.get("/api/reservas", function (req, res) {
    const reservations = [
        { checkIn: "2026-11-10",
          checkOut: "2026-11-15" }
    ];
    res.json(reservations);
});
```

Para recordar

Una ruta relaciona una petición con una respuesta. El mensaje inicial y la lista de reservas son dos respuestas distintas.

Comprobá que lo entendiste

¿Por qué el archivo del servidor se ejecuta con Node y no con una etiqueta script en la página?

11 / De la API al calendario

La web se abre desde el puerto tres mil, servido por Express. Su JavaScript hace una petición fetch a la dirección de reservas. Al usar una dirección relativa, consulta el mismo servidor que entregó la página.

Una función async puede usar await para esperar operaciones dentro de esa función. No bloquea toda la página. Primero llega la respuesta HTTP; después response.json interpreta su contenido. response.ok indica si el código de estado representa éxito.

La primera prueba mostraba los datos en la consola. Luego reemplazamos la lista local por una variable vacía y asignamos los datos recibidos. Hoy la lista ficticia está solo en el servidor; el calendario consume su respuesta.

Mientras carga, el calendario permanece oculto. Solo se dibuja y muestra después de validar que llegó una lista con fechas válidas y orden correcto. Si la petición o los datos fallan, aparece un mensaje de error. No se muestra una lista vacía como si todas las noches estuvieran libres.

También corregimos el evento que se ejecuta al mostrar la página: no debe borrar un mensaje de carga o de error. Las pruebas verifican que una reserva diferente enviada por el servidor modifica el calendario. Eso demuestra que ya no usa la copia anterior.

```
const response = await fetch("/api/reservas");
if (!response.ok) {
    throw new Error("Falló la consulta");
}
const data = await response.json();
// Validar la forma y las fechas antes de usarla.
reservations = data;
renderCalendar();
```

Para recordar

Una respuesta vacía válida no es lo mismo que un fallo de red. En el fallo, no debemos afirmar disponibilidad.

Comprobá que lo entendiste

¿Qué hace la asignación reservations igual data, y por qué la variable usa let?

12 / Repasar y seguir

El recorrido actual es: el navegador pide la página a Express, carga sus archivos, solicita las reservas a la API y dibuja el calendario con la respuesta. La persona elige fechas; la página comprueba el rango y muestra las noches. No se envía una solicitud para crear una reserva.

Pasaron dieciséis pruebas automáticas de reglas y selección. Cubren superposiciones, límites, años bisiestos, cambios de mes y fallos de carga simulados. Son evidencia sobre esos casos, no una garantía de que todo un sistema de producción esté terminado.

Para estudiar, explicá sin mirar la diferencia entre HTML, CSS y JavaScript. Después narrá cómo viaja una petición y cómo vuelve la respuesta. Por último, resolvé un ejemplo de superposición antes de mirar la fórmula.

Lo próximo del backend será trasladar las comprobaciones decisivas al servidor y luego guardar datos. Para evitar una doble reserva real, la base de datos y el servidor deberán resolver también los intentos simultáneos. Comprobar solo en el navegador no alcanza.

Quedan para etapas posteriores la creación y confirmación de reservas, PostgreSQL, el panel del dueño, permisos de acceso, cancelaciones completas y pagos. La mejora del contraste visual queda pendiente por pedido del usuario. El calendario actual es una base para seguir aprendiendo.

```
# Iniciar el servidor:
node server.js

# Ejecutar las pruebas actuales:
node --test tests/availability.test.cjs tests/calendar.test.cjs
```

Para recordar

Antes de dar algo por terminado, preguntá: ¿qué comprobé, con qué datos y qué todavía falta?

Comprobá que lo entendiste

¿Por qué dos personas podrían ver una fecha libre al mismo tiempo aunque el calendario funcione correctamente?